



Autonomiczny Robot Magazynowy

Grzegorz Lademann, Jakub Lidzbarski

Projekt zakłada

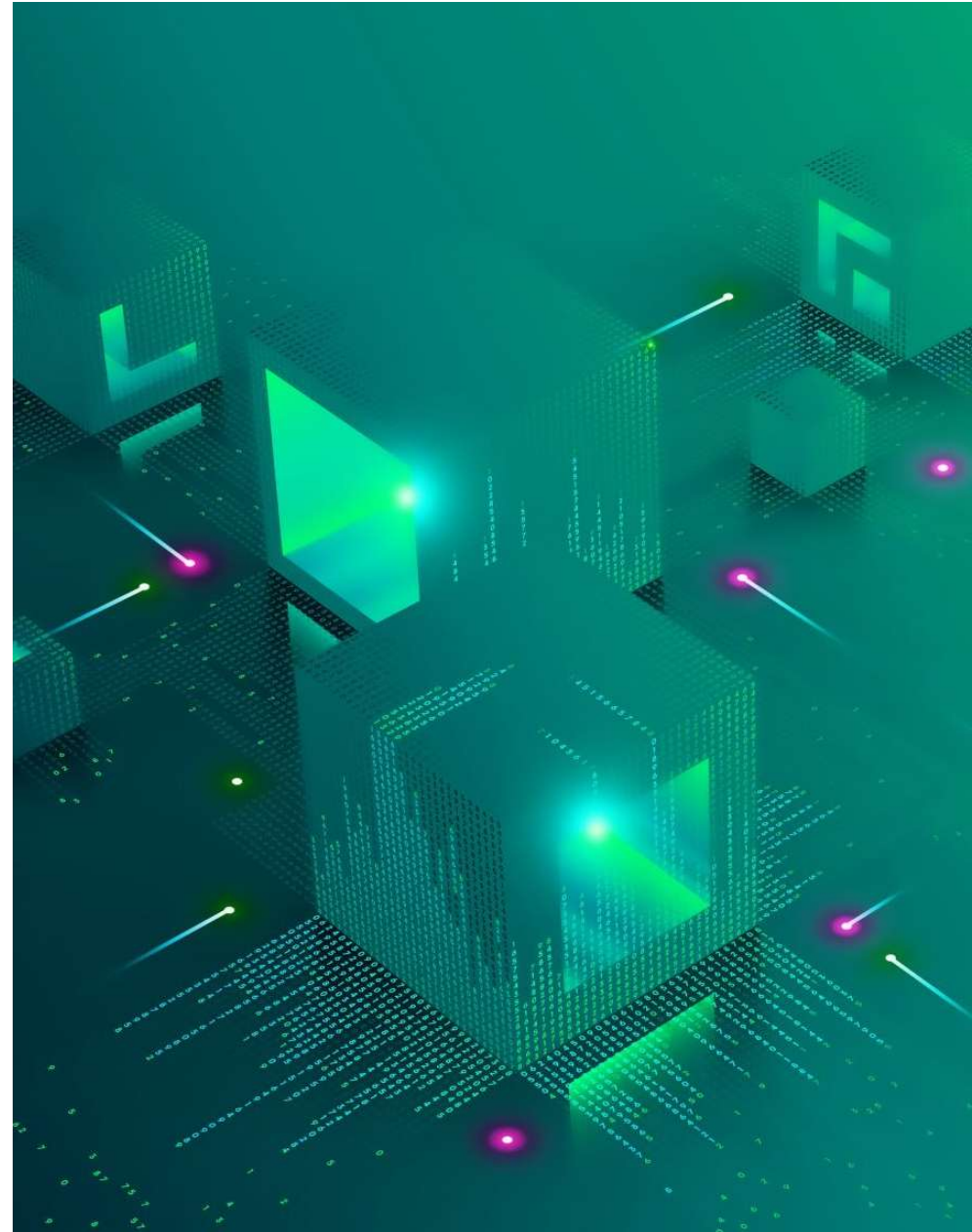
1. Budowę autonomicznego robota przystosowanego do poruszania się w przestrzeni magazynowej.

2. Opracowanie oprogramowania serwerowego, które będzie:

- Sterować robotem,
- Zbierać i zapamiętywać informacje o przestrzeni,
- Planować trasy przejazdu z uwzględnieniem omijania przeszkód oraz stref niedostępnych.
- Wymieniać dane między węzłami

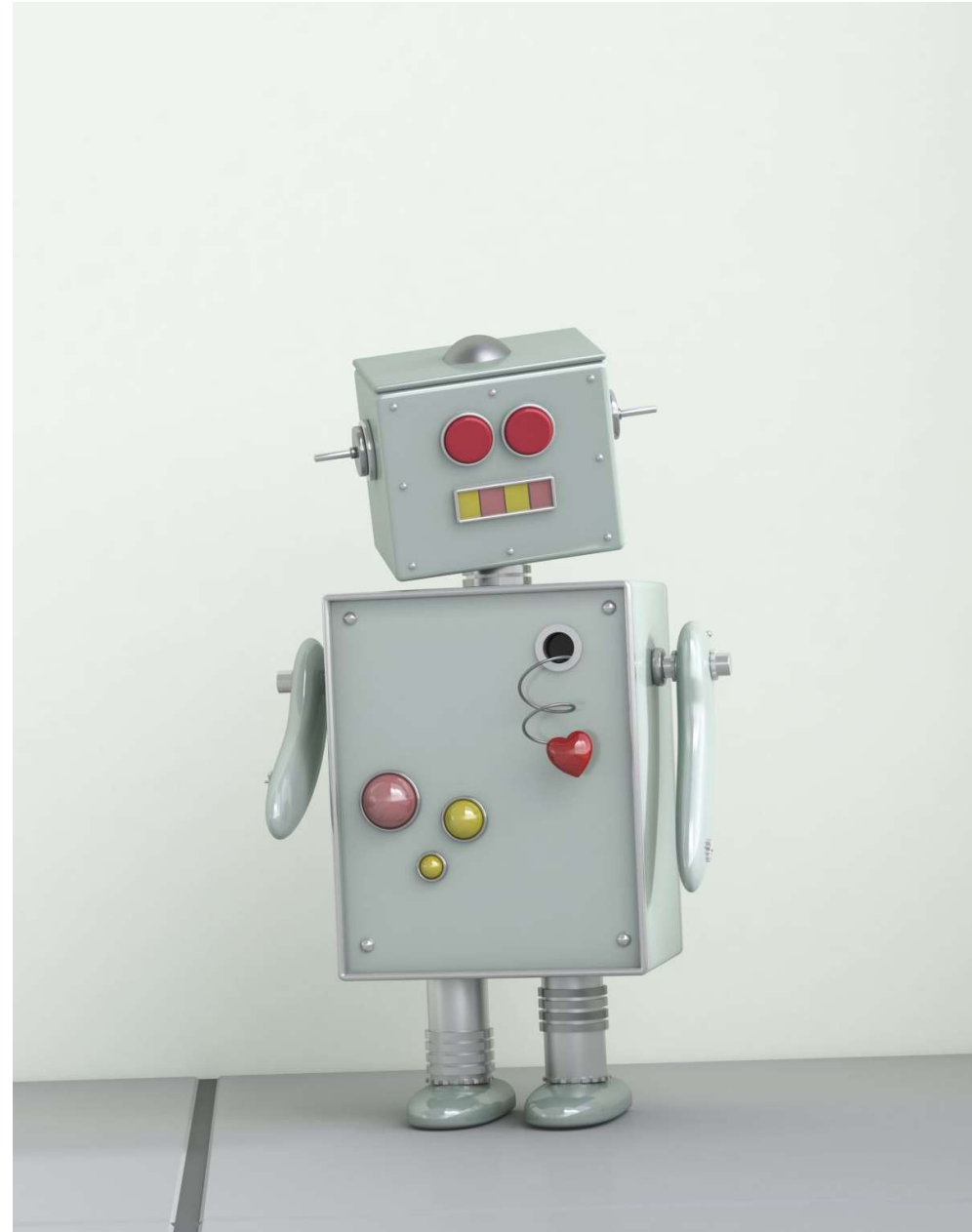
3. Przygotowanie aplikacji w Unreal Engine, której zadaniem będzie:

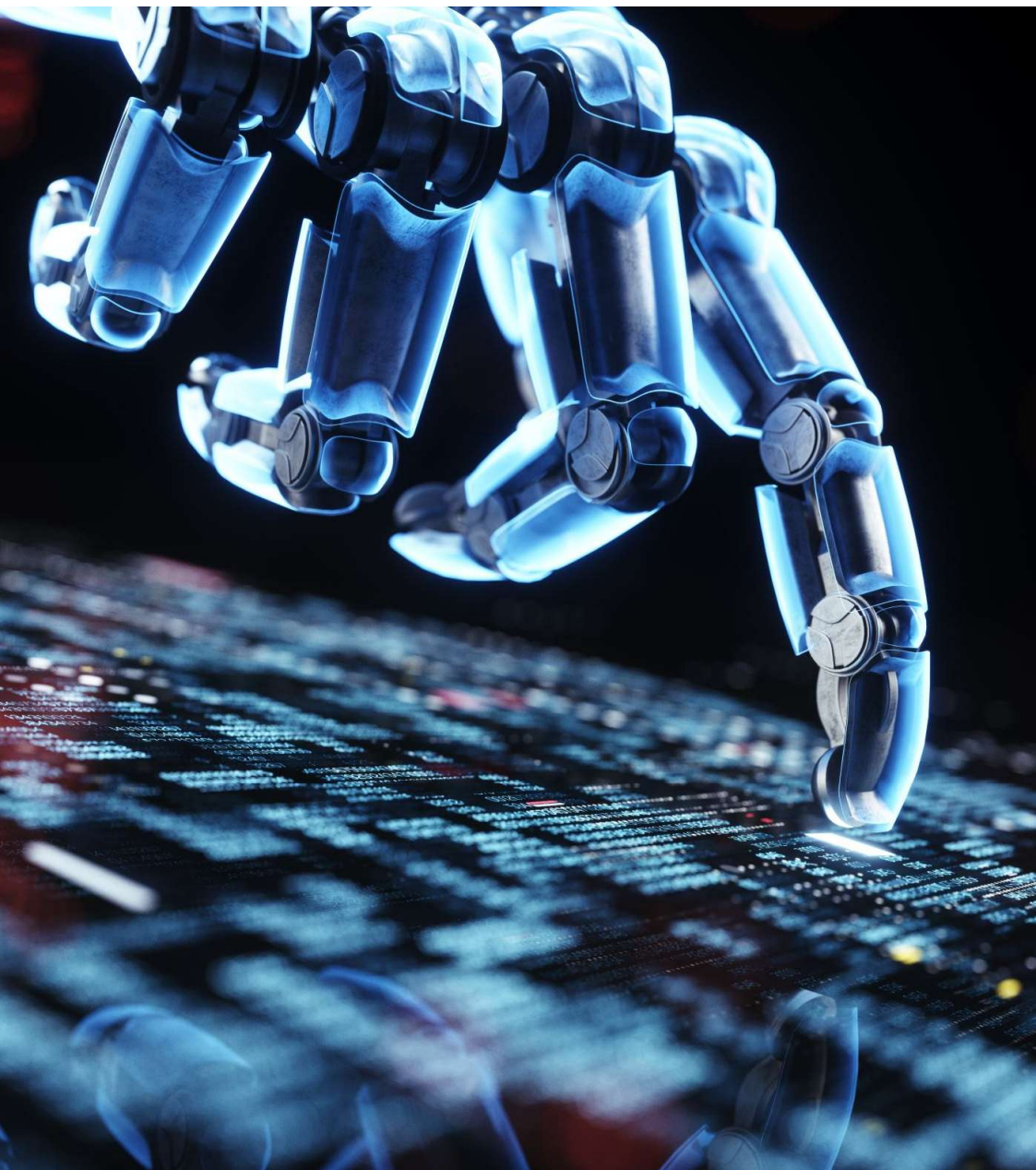
- wizualizacja zeskanowanej przestrzeni magazynowej,
- odwzorowanie ruchu robota,
- symulacja transportu przesyłek.



Etapy realizacji projektu

1. Wizualizacja robota i opracowanie modelu 3D
2. Zakupienie niezbędnych komponentów
3. Wydruk i montaż konstrukcji robota
4. Nauka i implementacja algorytmów (SLAM, ROS, Dijkstra)
5. Oprogramowanie serwera i systemu robota
6. Stworzenie aplikacji wizualizacyjnej w Unreal Engine
7. Testy jednostkowe i funkcjonalne
8. Prezentacja końcowa projektu





Ograniczenia

- System ograniczony do kontroli nad jednym robotem
- Maksymalny wymiar przestrzeni to 30x30m
- Maksymalna prędkość robota do 1km/h
- Wydajność aplikacji 3D
- Koszty
- Wielkość wydruku

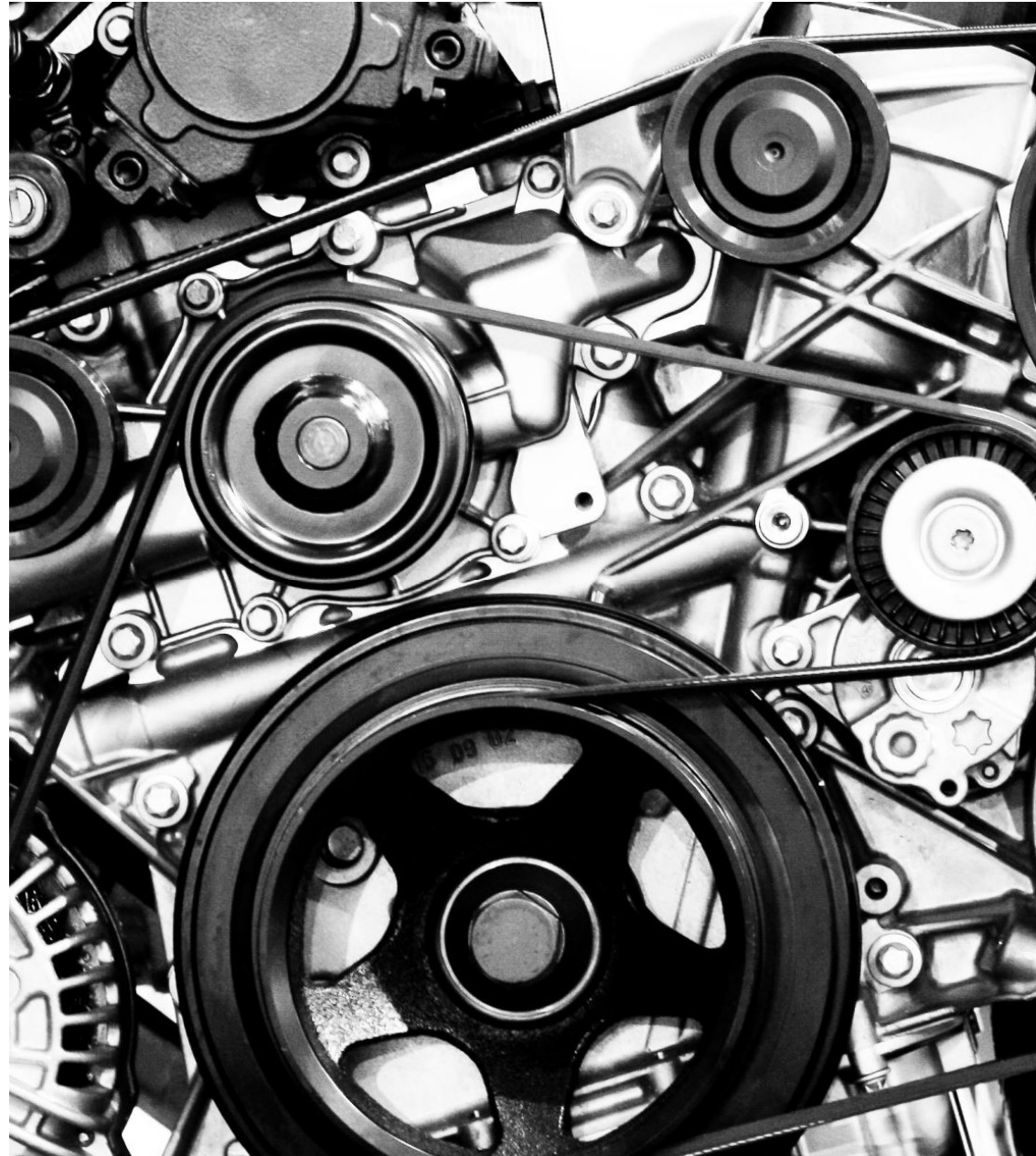
Założenia

- Opracowanie aplikacji 3D w silniku Unreal Engine
- Oprogramowanie serwera w języku Python
- Oprogramowanie sterujące (serwera) jest scentralizowane na jednym urządzeniu
- Oprogramowanie mikrokontrolera robota w języku C++
- Komunikacja między węzłami systemu poprzez Websocket
- Wykorzystanie algorytmów SLAM, ROS, DIJKSTRA
- Konfiguracja ustawień sieciowych mikrokontrolera poprzez port szeregowy
- Skanowanie z wykorzystaniem technologii Lidar
- Generowanie i aktualizowanie mapy w Unreal Engine
- Sterowanie ruchem robota przy pomocy dwóch niezależnych silników krokowych
- Ustawianie celu oraz bazy robota w środowisku Unreal Engine oraz synchronizacja ich z rzeczywistymi współrzędnymi



Wykorzystane Oprogramowanie

- Unreal Engine 5.6.1
- Arduino IDE 2.3.6
- Pycharm Community
- Blender 4.3



Spis Elementów

- **Komponenty Konstrukcyjne Robota**

- 2 szt. Silników krokowych NEMA 17
- 2 szt. Sterowników silników krokowych TMC2208
- 1 szt. 10DoF IMU GY-87
- 1 szt. Logic Converter
- 1 szt. Czujnik ToF VL53LO
- 1 szt. Mikrokontroler ESP32-S3-WROOM-1 (Lolin S3)
- 1 szt. Lidar 2D Delta-2A
- 2 szt. Kątowniki NEMA 17
- Wydrukowane Elementy Plastikowe
- Diody LED THT 3mm w kolorach: biały, czerwony, pomarańczowy
- 4 szt. Baterie akumulatorowe 18650
- 2 szt. Koszyk na akumulatory 18650
- 1 szt. Wyświetlacz segmentowy pokazujący stan naładowania akumulatorów
- 2 szt. Uniwersalne kółka obrotowe
- 2szt. Opona z gumy PU o średnicy 63mm
- 2 szt. Serwo SG-90

- **Sieć**

- Mikrotik hap ac2

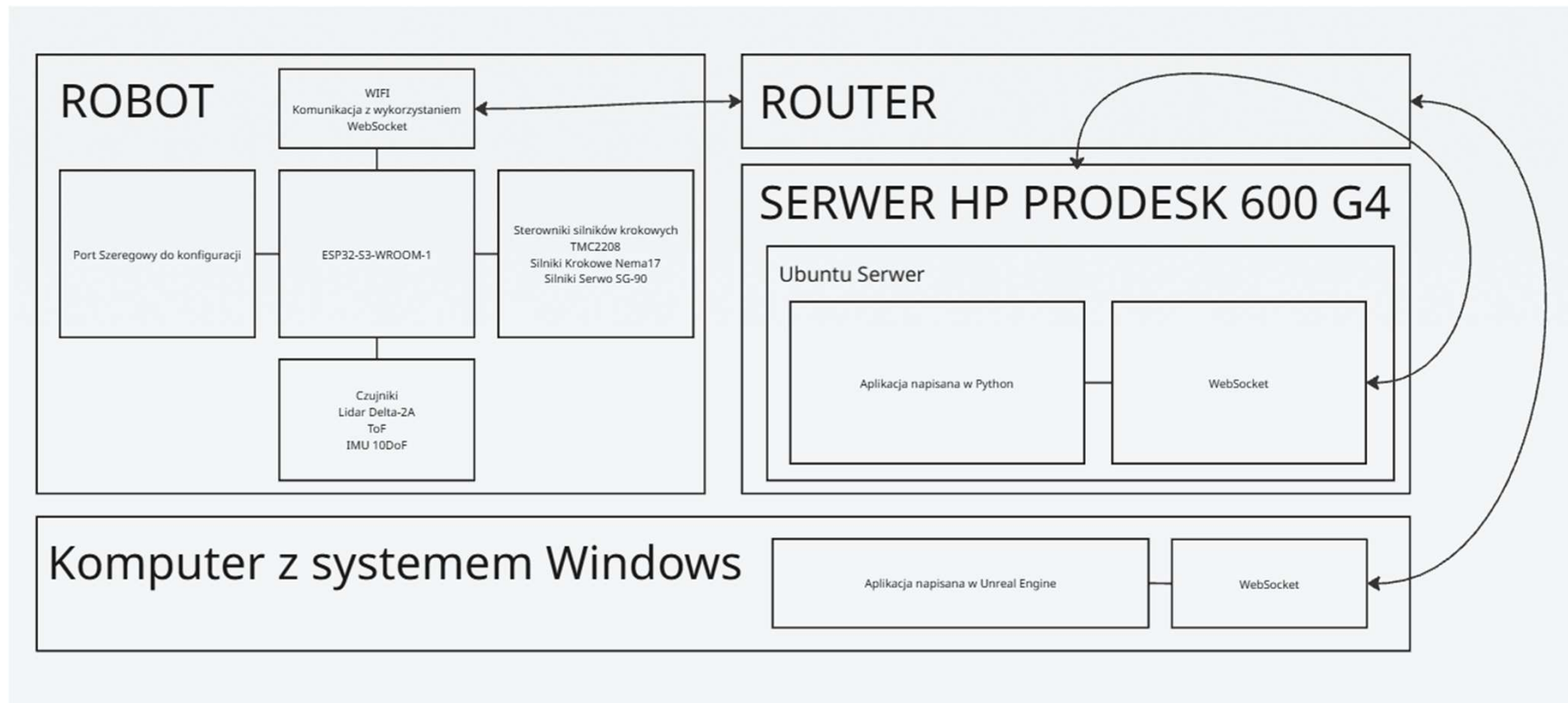
- **Serwer**

- HP Prodesk 600 G4 – Ubuntu Server 24.04 LTS

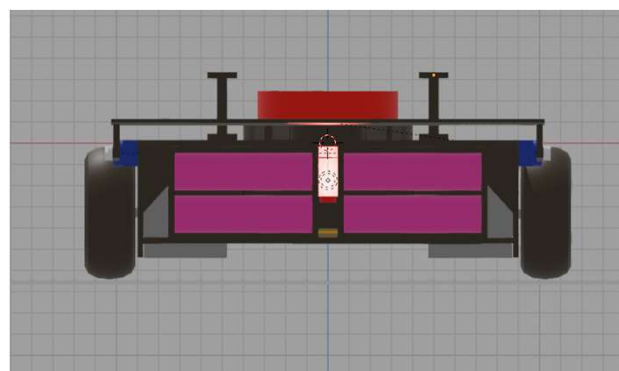
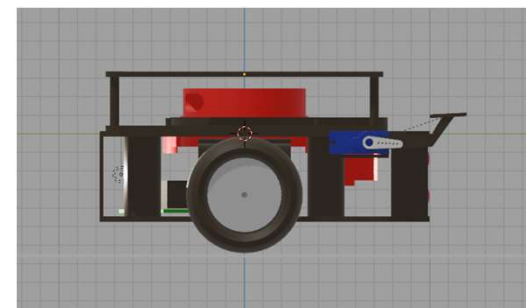
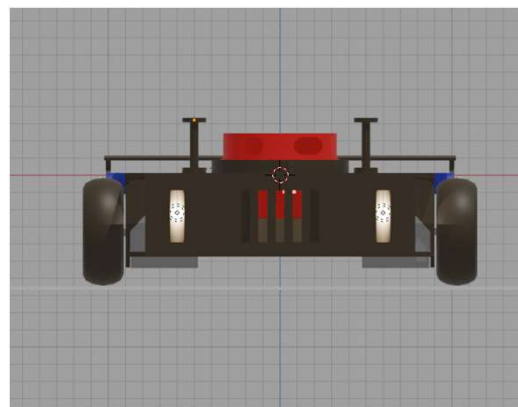
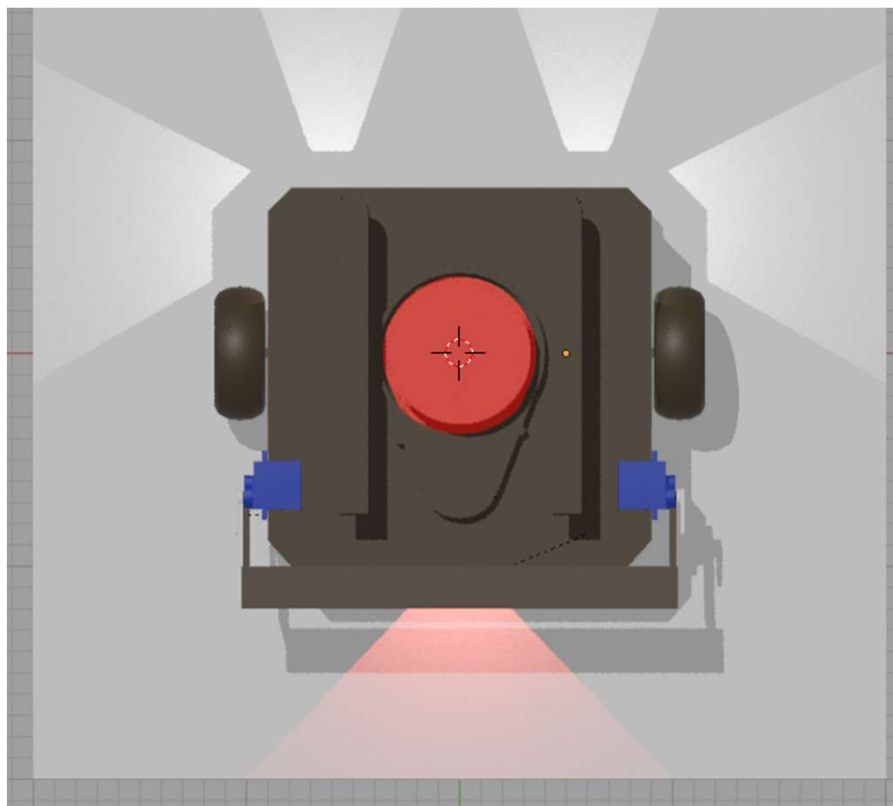
- **Komputer**

- Z systemem Windows oraz aplikacją Unreal Engine

Komponenty systemu



Wizualizacja Robota



Websocket

- Wszystkie dane będą przechowywane na serwerze, a komunikacja z nim będzie odbywać się za pomocą WebSocketu. W UnrealEngine będziemy obsługiwać różne komendy wysyłane z serwera, tak samo jak UnrealEngine będzie wysyłał różne komendy. Dane przesyłane do Unreala to min. Do tworzenia mapy, ustawiania robota na mapie 2D a komendy z naszego środowiska graficznego to np. Przekazywanie informacji o docelowej pozycji, do której powinien przemieścić się fizyczny robot.
-

```
!robot_data;57,-70,315,315,315,315,315,315;atad_torob!  
!robot_data;-60,-42,19,157,143,280,215;atad_torob!  
!robot_data;-25,-72,185,150,337,289,202;atad_torob!  
!robot_data;-25,-72,185,150,337,289,202;atad_torob!  
!robot_data;-25,-72,185,150,337,289,202;atad_torob!  
!robot_data;82,69,257,177,120,286,270;atad_torob!  
!robot_data;82,69,257,177,120,286,270;atad_torob!  
!robot_data;82,69,257,177,120,286,270;atad_torob!  
!robot_data;-92,-33,85,34,25,219,191;atad_torob!  
!robot_data;-92,-33,85,34,25,219,191;atad_torob!  
!robot_data;-92,-33,85,34,25,219,191;atad_torob!  
!robot_data;-83,38,221,50,257,149,224;atad_torob!  
!robot_data;-83,38,221,50,257,149,224;atad_torob!  
!robot_data;-83,38,221,50,257,149,224;atad_torob!  
!robot_data;-83,38,221,50,257,149,224;atad_torob!  
!robot_data;2,-65,116,129,288,189,128;atad_torob!  
!robot_data;2,-65,116,129,288,189,128;atad_torob!  
!robot_data;2,-65,116,129,288,189,128;atad_torob!  
!robot_data;97,-55,200,223,297,15,28;atad_torob!  
!robot_data;97,-55,200,223,297,15,28;atad_torob!  
!robot_data;97,-55,200,223,297,15,28;atad_torob!  
!robot_data;96,-25,315,1,167,207,302;atad_torob!  
!robot_data;96,-25,315,1,167,207,302;atad_torob!  
!robot_data;96,-25,315,1,167,207,302;atad_torob!  
!robot_data;96,-25,315,1,167,207,302;atad_torob!  
!robot_data;-67,72,253,146,99,247,113;atad_torob!  
!robot_data;-67,72,253,146,99,247,113;atad_torob!  
!robot_data;-67,72,253,146,99,247,113;atad_torob!  
!robot_data;12,-11,357,255,170,153,23;atad_torob!  
!robot_data;12,-11,357,255,170,153,23;atad_torob!  
!robot_data;12,-11,357,255,170,153,23;atad_torob!  
!robot_data;79,-27,99,236,76,8,76;atad_torob!
```

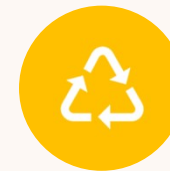
Generowanie Mapy 2D



Generowanie mapy w Unreal Engine polega na pobieraniu danych z lidara na temat porozmieszczanych przedmiotów po pomieszczeniu.



Po sczytaniu danych tworzymy tablice liczb gdzie każda liczba reprezentuje poszczególną informację na temat stanu naszego pokoju



0 – zielona - przestrzeń wolna



1 - żółta - przestrzeń zajęta



2 – czerwone - przestrzeń nie odkryta



Każdy punkt w tablicy jest odzwierciedleniem 5cm w naszym pokoju.

Wizualizacja mapy 2D pokoju

